

System Specification: Go-based Algorithmic Backtesting Engine

1. Projektöversikt

Målet med projektet är att bygga en högpresterande, modulär och parallelliserad backtesting-motor för algoritmisk handel i Go. Systemet är designat för att processa stora mängder historisk börsdata (High-Frequency Trading data), exekvera polymorfa handelsstrategier och utvärdera prestanda med minimalt minnesavtryck och maximal exekveringshastighet.

2. Systemarkitektur

Arkitekturen bygger på principen om "Separation of Concerns" för att säkerställa att systemet är testbart och utbyggbart. Koden är uppdelad i följande huvudpaket:

- **types:** Innehåller systemets globala datastrukturer (Domain Models). Centraliserar beroenden så att andra paket inte korsrefererar varandra.
- **data:** Ansvarar för I/O-operationer, filhantering och minnesallokering av rådata.
- **strategy:** Innehåller all affärslogik för handel, inkapslat bakom ett gemensamt interface.
- **engine:** Kärnmotorn som hanterar simulering av tid, beräkning av tekniska indikatorer och framöver hantering av Goroutines för parallell exekvering.

Aha, då är jag helt med! Du vill att vi utgår från *exakt* den texten du klistrade in och bara fyller på med de saknade typerna (MarketData som du påpekade, plus de nya för parallelliseringen), så att listan blir komplett.

Här är det uppdaterade och kompletta stycket, redo att klistras in i er specifikation:

3. Datamodeller (types/models.go)

För att optimera prestanda och minnesanvändning under simuleringen (for-loopen) är datamodellerna designade med primitiva datatyper där det är möjligt.

- **Bar:** Representerar en specifik tidsenhet av marknadsdata (OHLCV).
 - *Optimering:* Tidsstämplar (Timestamp) hanteras strikt som int64 (Unix Epoch) i databasen och kärnmotorn för $O(1)$ jämförelser och för att undvika I/O-flaskhalsar associerade med time.LoadLocation i Go's inbyggda tidsbibliotek.
- **Signal:** Bär strategins beslut (BUY, SELL, HOLD) tillsammans med metadata (Stop Loss, Take Profit, Risk/Reward).
- **Trade:** Ett genomfört avslut (transaktion) som loggas för senare prestationsanalys.

- **MarketData:** Används för inkommande JSON-strömmar (t.ex. live-data eller API-anrop). Håller koll på specifika Ticker-namn, nuvarande pris och aktuellt glidande medelvärde (SMA).
- **OptimizationTask:** Paketerar ett unikt simuleringsjobb (en specifik uppsättning parametrar och en vald strategi) som distribueras till en isolerad worker (Goroutine) för parallell exekvering.
- **PerformanceMetrics:** Det matematiska resultatet (Net Profit, Win Rate, Max Drawdown) av en avslutad OptimizationTask, vilket returneras för att utvärdera strategins effektivitet.

4. Datahantering (data/storage.go)

Systemet använder formatet **Apache Parquet** via github.com/segmentio/parquet-go.

- **Lässtrategi:** Datan läses in i batchar (buffertar) för att minska antalet Garbage Collection (GC) sweeps.
- **Dataintegritet:** LoadData-funktionen garanterar kronologisk integritet genom att tillämpa en in-place sortering (sort.Slice) baserat på Unix-tid innan datan returneras till huvudprogrammet. Tidskomplexitet: $\mathcal{O}(N \log N)$.

5. Strategimönster (Strategy Pattern)

Systemet implementerar designmönstret *Strategy* via [strategy.go](https://github.com/robp23/strategy-go) för att uppnå full polymorfism.

- **Strategy Interface:** Definierar kontraktet (Name() string, OnBar(bar types.Bar, history []types.Bar) types.Signal) som alla strategier måste uppfylla.
- **Nuvarande moduler:**
 1. **SMACrossStrategy:** Ett trendföljande system som använder glidande medelvärden och volymbekräftelse, samt swing high/low för stop-loss placering.
 2. **FVGStrategy:** En algoritm baserad på "Fair Value Gaps" som identifierar obalanser i marknaden över definierade tidsfönster och exekverar trades när priset återtestar (retest) zonen i riktning med den övergripande trenden.

6. Kärnmotor (engine/backtest.go & metrics.go)

- **SimulateTicker:** Går igenom historisk data sekventiellt och matar Strategy-interfacet med nuvarande Bar och en slice av föregående history.
- *Minnesoptimering:* Historik genereras dynamiskt i loopen (`data[:i+1]`) genom att flytta slice-pekare istället för att kopiera data till nya arrayer, vilket håller tidskomplexiteten för minnesallokering till $\mathcal{O}(1)$ per tick.
- **metrics.go:** Ett frikopplat bibliotek av rena matematiska funktioner (SMA, AvgVolume, SwingHigh/Low) som anropas av strategierna.

7. Kommande Implementationer (Issues)

1. **Concurrency (Worker Pool):** För att hantera parameteroptimering ("Generate parameter combinations") ska en "Worker Pool" implementeras med Goroutines. Huvudtråden (main) distribuerar olika instanser av strategier (t.ex. SMA 50/200, SMA 20/50) via "Channels" till lediga workers för parallell backtesting.
2. **Task Model:** Designa en struct (OptimizationTask) som paketerar en uppsättning parametrar och en dedikerad slice med testdata för en specifik worker att exekvera autonomt utan att orsaka "Race Conditions" mot andra goroutines.
3. **Performance Metrics:** Implementera en modul som aggregerar listan av []types.Trade för att beräkna Sharpe Ratio, Maximum Drawdown och Win Rate.